Einführung in das LSP

From Scratch mit Go und Tree-sitter

Was macht ein Language Server?

Language Server bieten eine Reihe von “language intelligence tools“,

darunter:

- Code completion
- Hover Informationen
- Go to definition
- ...

Der Editor fungiert als Client und sendet Anfragen an den Server

Proof-of-Concept: "FlowLang"

Eine einfache Sprache zur Definition von Datenverarbeitungs-Pipelines.

Sie definiert, woher die Daten stammen (Quelle):

```
source “raw_user_data“ {
    path: “/data/users.csv“
}
```

welche Transformationen (Task) darauf angewendet werden:

```
task „filter_active_users“ {
    input: “raw_user_data“
    transformer: “status == ‘active‘“
}
```

und wohin sie fließen (Sink):

```
sink „active_user_report“ {
    input: “filter_active_users“
    path: “/active_users.json“
}
```

Der Stack

Go (Golang)

Language Server

Modern, einfach, performant

Tree-sitter

Parser, Queries

Parsed den Code zu einem Concrete Syntax Tree (CST) innerhalb von Millisekunden. Bietet eine eigene Querysprache um Codeinhalte zu extrahieren

CST

```
(file [0, 0] - [19, 0]
  (comment [0, 0] - [0, 27])
  (source_definition [1, 0] - [4, 1]
    name: (string_literal [1, 7] - [1, 22])
    (source_body [1, 23] - [4, 1]
      (source_body_path [2, 4] - [2, 27]
        path: (string_literal [2, 10] - [2, 27]))
      (key_value_pair [3, 4] - [3, 21]
        key: (identifier [3, 4] - [3, 12])
        value: (string_literal [3, 14] - [3, 21]))))
  (task_definition [7, 0] - [11, 1]
    name: (string_literal [7, 5] - [7, 27])
    (task_body [7, 28] - [11, 1]
      ...
      ...))
  (sink_definition [14, 0] - [18, 1]
    name: (string_literal [14, 5] - [14, 25])
    (sink_body [14, 26] - [18, 1]
      ...
        path: (string_literal [17, 10] - [17, 38]))))
```

Code

```
# Definiert die Datenquelle
source "raw_user_data" {
    path: "/data/users.csv"
    encoding: "utf-8"
}

task "filter_active_useras" {
    ...

}

sink "active_user_report" {
    ...
    path:
"/reports/active_users.json"
}
```

Der LSP Standard

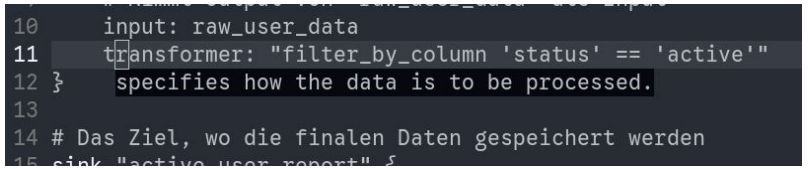
Ein standardisiertes, JSON-RPC-basiertes Protokoll, das die Kommunikation zwischen Code-Editoren Language Servern ermöglicht

Es gibt verschiedene Methoden, die der Client auf dem Server aufrufen kann:

textDocument/completion, textDocument/hover, textDocument/signatureHelp, textDocument/declaration, textDocument/definition, textDocument/typeDefinition, textDocument/implementation, textDocument/references...

Feature: Hover

Annahme: Neovim hat den Language Server als separaten Prozess gestartet und kommuniziert mit ihm über Standard-Input/Output.
Der Server hat außerdem das Dokument in irgendeiner Form zwischengespeichert (textDocument/didOpen)



1. Trigger

Hover über einem Textelement ausgelöst

2. Node-Typ ermitteln

(serverseitig) Feststellen über welcher Art Node gehovert wurde

3. Informationen empfangen

Zugehörige Textinformationen empfangen und anzeigen

```
 8 task "filter_active_users"
 9     # Nimmt Output von "ra
10     input: raw_user_data
11     transformer: "filter_b
12 }
13
```

```
 8 task "filter_active_users"
 9     # Nimmt Output von "ra
10     input: raw_user_data
11     transformer: "filter_b
12 }
13
```

hover

```lua
function M.hover(config)
  validate('config', config, 'table', true)

  config = config or {}
  config.focus_id = 'textDocument/hover'

  lsp.buf_request_all(0, 'textDocument/hover', client_positional_params(), function(results, ctx)
    local bufnr = assert(ctx.bufnr)
    if api.nvim_get_current_buf() ~= bufnr then
      -- Ignore result since buffer changed. This happens for slow language servers.
      return
    end

    -- Filter errors from results
    local results1 = {} --- @type table<integer,lsp.Hover>
    local empty_response = false

    for client_id, resp in pairs(results) do
      local err, result = resp.err, resp.result
      if err then
```

https://github.com/neovim/neovim/blob/d6be2b33125d6706779beed7f8e83015bdff9396/runtime/lua/vim/lsp/buf.lua#L48

```
 8 task "filter_active_users"
 9     # Nimmt Output von "ra
10     input: raw_user_data
11     transformer: "filter_b
12 }
13
```

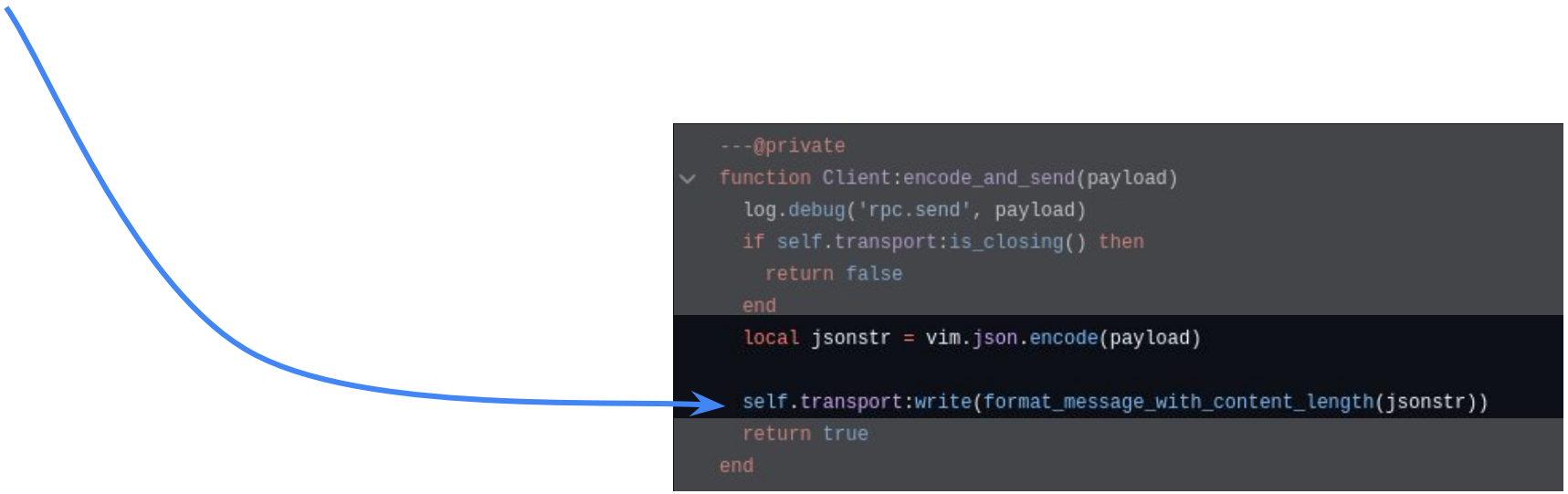
hover

```lua
function M.hover(config)
  validate('config', config, 'table', true)

  config = config or {}
  config.focus_id = 'textDocument/hover'

  lsp.buf_request_all(0, 'textDocument/hover', client_positional_params(), function(results, ctx)
    local bufnr = assert(ctx.bufnr)
    if api.nvim_get_current_buf() ~= bufnr then
      -- Ignore result since buffer changed. This happens for slow language servers.
      return
    end

    -- Filter errors from results
    local results1 = {} --- @type table<integer,lsp.Hover>
    local empty_response = false

    for client_id, resp in pairs(results) do
      local err, result = resp.err, resp.result
      if err then
```

https://github.com/neovim/neovim/blob/d6be2b33125d6706779beed7f8e83015bdff9396/runtime/lua/vim/lsp/buf.lua#L48

```json
{
  "jsonrpc": "2.0",
  "id": 123,
  "method": "textDocument/hover",
  "params": {
    "textDocument": {
      "uri": "file:///pfad/zur/datei.flow"
    },
    "position": {
      "line": 10,
      "character": 5
    }
  }
}
```

```
Content-Length: 189

{
  "jsonrpc": "2.0",
  "id": 123,
  "method": "textDocument/hover",
  "params": {
    "textDocument": {
      "uri": "file:///pfad/zur/datei.flow"
    },
    "position": {
      "line": 10,
      "character": 5
    }
  }
}
```



```lua
---@private
function Client:encode_and_send(payload)
  log.debug('rpc.send', payload)
  if self.transport:is_closing() then
    return false
  end
  local jsonstr = vim.json.encode(payload)

  self.transport:write(format_message_with_content_length(jsonstr))
  return true
end
```

https://github.com/neovim/neovim/blob/d6be2b33125d6706779beed7f8e83015bdff9396/runtime/lua/vim/lsp/rpc.lua#L261

```go
reader := bufio.NewReader(os.Stdin)

// main read loop
for {
  contentLength, err := readContentLength(reader)
  if err != nil {
    if err == io.EOF {
      log.Println("client connection closed")
      return
    }
    log.Printf("failed to read header: %v", err)
    continue
  }

  body := make([]byte, contentLength)
  _, err = io.ReadFull(reader, body)
  if err != nil {
    log.Printf("failed to read body: %v", err)
    continue
  }

  log.Printf("received: %s", string(body))
  handleMessage(body)
}
```

```go
case "initialize":
  log.Println("client 'initialize' received")

  response := InitializeResult{
```

```go
case "initialized":
  log.Println("'initialized' received")

case "textDocument/didOpen":
  log.Println("'textDocument/didOpen' received")

  var params DidOpenParams
```

```go
case "textDocument/didChange":
  log.Println("'textDocument/didChange' received")

  var params DidChangeParams
```

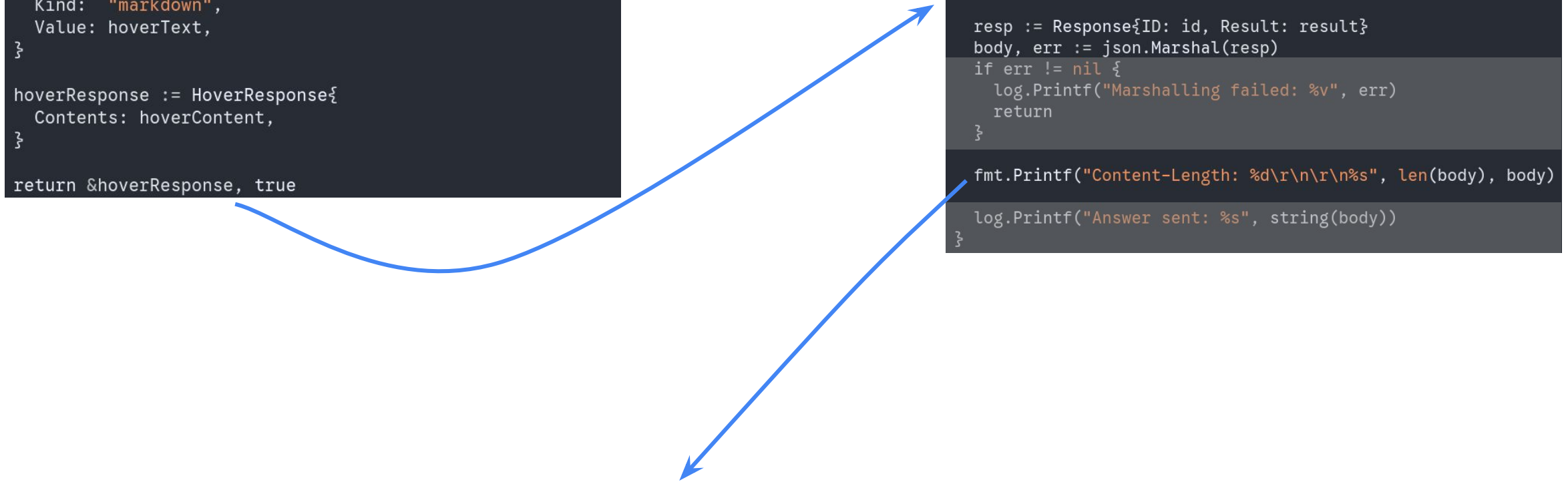
```go
case "textDocument/hover":
  log.Println("'textDocument/hover' received")

  var params HoverParams
  if err := json.Unmarshal(*request.Params, &params); err != ni
    log.Printf("failed to parse HoverParams: %v", err)
    sendResponse(request.ID, nil)
    return
  }
  hoverResponse, ok := hoverFeature(params.TextDocument, params
  if !ok {
    sendResponse(request.ID, nil)
    return
  }
```

```
Content-Length: 189


{
...
  "method": "textDocument/hover",
...
...
}
```

```
func hoverFeature(textDocument TextDocumentIdentifier,position     Position) (*HoverResponse,bool){
  doc, ok := getDocument(textDocument.URI)
  if !ok {
    log.Printf("document for hover not found: %s", textDocument.URI)
    return nil,false
  }

  tree := tsParser.Parse([]byte(doc), nil)
  rootNode := tree.RootNode()

  point := sitter.Point{Row: uint(position.Line), Column: uint(position.Character)}

  node := rootNode.NamedDescendantForPointRange(point, point)

  if node == nil {
    log.Printf("no node found at %v:%v", position.Line, position.Character)
    return nil,false
  }

  log.Printf("node kind: %s", node.Kind())

  hoverText := ""
  switch node.Kind() {
  case "source_definition":
    hoverText = "defines a **source**."
  case "source_body_path":
    hoverText = "specifies the file path from which the data should be read."
  case "task_definition":
    hoverText = "defines a **task** in the pipeline."
```

```
hoverContent := MarkupContent{
  Kind:  "markdown",
  Value: hoverText,
}

hoverResponse := HoverResponse{
  Contents: hoverContent,
}

return &hoverResponse, true
```

```
func sendResponse(id *json.RawMessage, result any) {

  resp := Response{ID: id, Result: result}
  body, err := json.Marshal(resp)
  if err != nil {
    log.Printf("Marshalling failed: %v", err)
    return
  }

  fmt.Printf("Content-Length: %d\r\n\r\n%s", len(body), body)

  log.Printf("Answer sent: %s", string(body))
}
```



```
Content-Length: 103


{
   "id":123,
   "result":{
      "contents":{
         "kind":"markdown",
         "value":"specifies how the data is to be processed."
      }
   }
}
```

```lua
--- @async
local function request_parser_loop()
  local buf = strbuffer.new()
  while true do
    local msg = buf:tostring()
    local header_end = msg:find('\r\n\r\n', 1, true)
    if header_end then
      local header = buf:get(header_end + 1)
      buf:skip(2) -- skip past header boundary
      local content_length = get_content_length(header)
      while strbuffer.len(buf) < content_length do
        buf:put(coroutine.yield())
      end
      local body = buf:get(content_length)
      buf:put(coroutine.yield(body))
    else
      buf:put(coroutine.yield())
    end
  end
end
```

https://github.com/neovim/neovim/blob/d6be2b33125d6706779beed7f8e83015bdff9396/runtime/lua/vim/lsp/rpc.lua#L193

```
Content-Length: 103

{
   "id":123,
...
}
```

```lua
while true do
  local ok, res = coroutine.resume(co, chunk)
  if not ok then
    on_error(res, M.client_errors.INVALID_SERVER_MESSAGE)
    break
  elseif res then
    handle_body(res)
    chunk = ''
  else
    break
  end
```

```lua
function M.hover(_, result, ctx, config)
  config = config or {}
  config.focus_id = ctx.method
  if api.nvim_get_current_buf() ~= ctx.bufnr then
    -- Ignore result since buffer changed. This happens for slow language servers.
    return
  end
  if not (result and result.contents) then
    if config.silent ~= true then
      vim.notify('No information available')
    end
    return
  end
  local format = 'markdown'
  local contents ---@type string[]
  if type(result.contents) == 'table' and result.contents.kind == 'plaintext' then
    format = 'plaintext'
    contents = vim.split(result.contents.value or '', '\n', { trimempty = true })
  else
    contents = util.convert_input_to_markdown_lines(result.contents)
  end
  if vim.tbl_isempty(contents) then
    if config.silent ~= true then
      vim.notify('No information available')
    end
    return
  end
  return util.open_floating_preview(contents, format, config)
end
```

https://github.com/neovim/neovim/blob/4e1644d4d3cb6b2ae7bd7110add8424e5217882b/runtime/lua/vim/lsp/handlers.lua#L383

```
10      input: raw_user_data
11      transformer: "filter_by_column 'status' == 'active'"
12  }     specifies how the data is to be processed.
13
14  # Das Ziel, wo die finalen Daten gespeichert werden
```

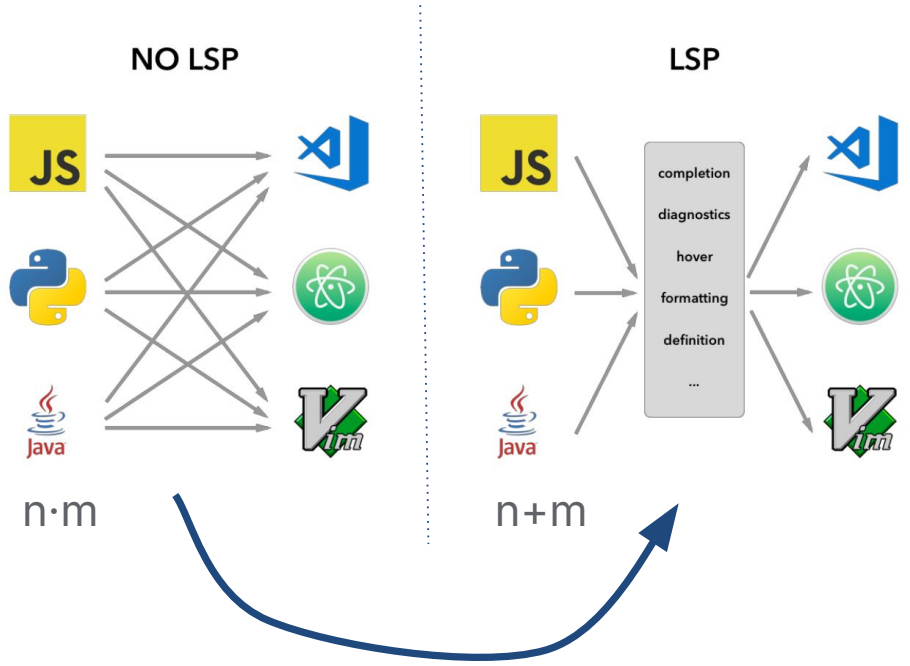
```
# Ein Task, der die Daten filtert
task "filter_active_users" {
    # Nimmt Output von "raw_user_data" als Input
    specifies how the data is to be processed.
    transformer: "filter_by_column 'status' == 'active'"      You,
```

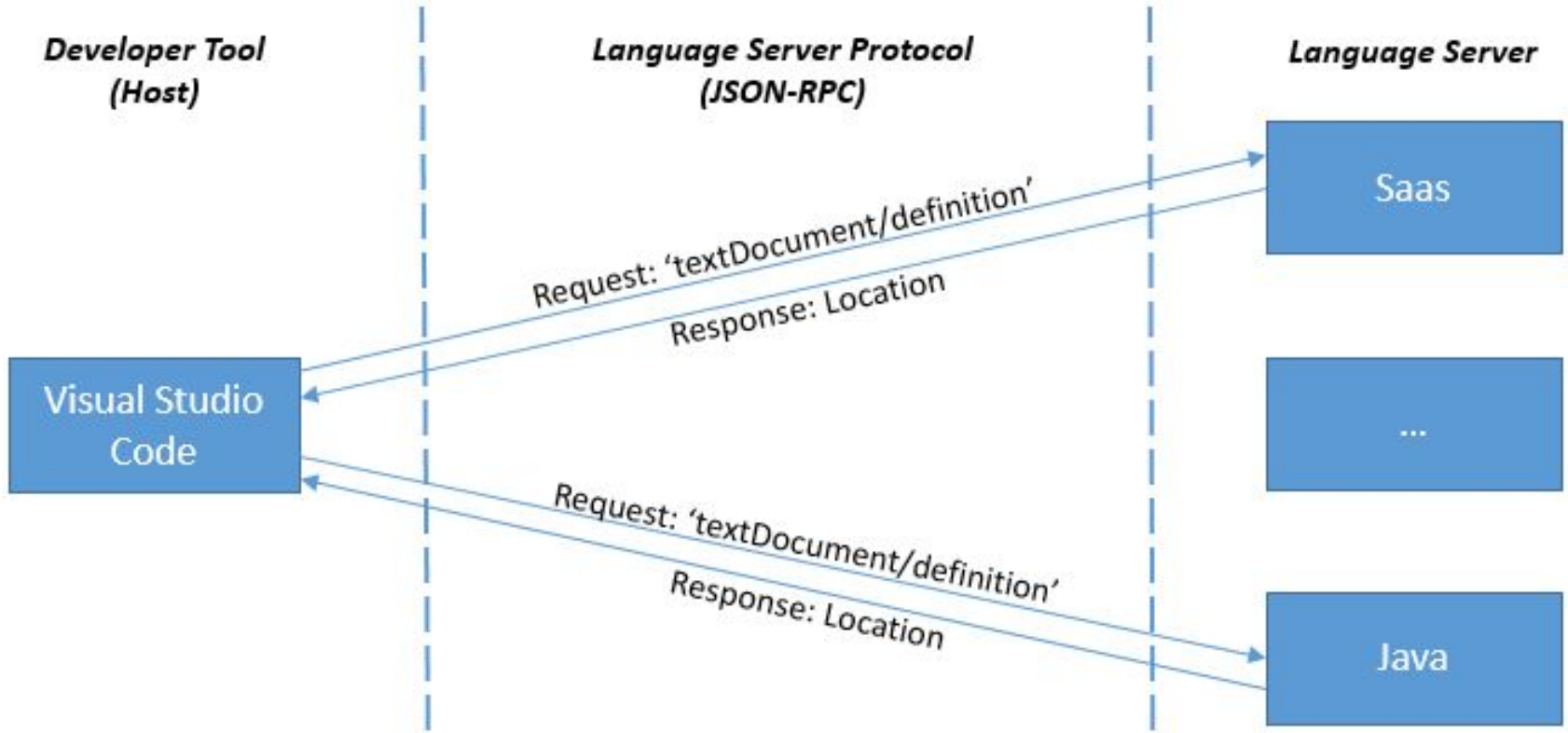
Vor LSP

In der Vergangenheit benötigte jeder Editor für jede Sprache ein individuelles Plugin

- Vim Python Plugin
- VS Code Go Extension
- Sublime Ruby Package
- ....

Folge: Hoher Aufwand und inkonsistente (Feature-) Implementierungen




Developer Tool
(Host)
Language Server Protocol
(JSON-RPC)
Language Server
Saas
...
Java
Visual Studio
Code
Request: 'textDocument/definition'
Response: Location
Request: 'textDocument/definition'
Response: Location

Ausblick

Fragen ?